



UNIVERSITY OF ASIA PACIFIC

DEPARTMENT OF CSE

Lab Report - 1

COURSE CODE: CSE 402.

COURSE TITLE: OPERATING SYSTEM LAB.

SUBMITTED BY:

NAME : Swagoto Utsab Singha Roy

STUDENT ID : 23101124

SEMESTER : 4th YEAR 1st SEMESTER

SECTION : C - 1

DATE : 13 - 08 - 2026

SUBMITTED TO
AITA RAHMAN ORTHI
LECTURER AT UAP

1. Problem:

Implement the **First Come First Serve (FCFS)** CPU Scheduling Algorithm for the following processes.

Process	Arrival Time (AT)	Burst Time (BT)
P0	3	5
P1	2	4
P2	4	3
P3	1	2
P4	5	3

Calculate CT, TAT, WT, AVG TAT and AVG WT. Also represent the process execution sequence.

2. Source Code:

```
1  lst = [  
2      ["P0",3,5],  
3      ["P1",2,4],  
4      ["P2",4,3],  
5      ["P3",1,2],  
6      ["P4",5,3]  
7  ]  
8  lst.sort(key=lambda x:x[1])  
9  ct = []  
10 tat = []  
11 wt = []  
12 current_time = 0  
13 for i in lst:  
14     name, at, bt = i  
15     if current_time < at:  
16         current_time = at  
17     current_time += bt  
18     ct.append(current_time)  
19     turnaround = current_time - at  
20     tat.append(turnaround)  
21     waiting = turnaround - bt  
22     wt.append(waiting)  
23 print(f"{'P_id':<6}{'AT':<6}{'BT':<6} {'CT':<6}{'TAT':<8}{'WT':<6}")  
24 for i in range(len(lst)):  
25     print(f"{'lst[i][0]:<6}{'lst[i][1]:<6}{'lst[i][2]:<6}{'ct[i]:<6}{'tat[i]:<8}{'wt[i]:<6}")  
26 avg_tat = sum(tat) / len(tat)  
27 print(f"\nAverage TAT: {avg_tat:.2f}")  
28 avg_wt = sum(wt) / len(wt)  
29 print(f"Average WT: {avg_wt:.2f}")
```

3. Output:

P_id	AT	BT	CT	TAT	WT
p3	1	2	3	2	0
p1	2	4	7	5	1
P0	3	5	12	9	4
p2	4	3	15	11	8
p4	5	3	18	13	10
Average TAT: 8.00					
Average WT: 4.60					

4. Discussion:

In this experiment, the FCFS CPU Scheduling Algorithm was implemented using Python for a new set of processes with arrival times and burst times as given. The processes were sorted according to their arrival time and each was executed in that order using `current_time` counter to compute the Completion Time (CT) of every process.

Process P3, being the first to arrive ($AT = 1$) started execution immediately and finished with zero waiting time. P1 arrived next ($AT = 2$) and began right after P3 completed at time 3, giving it a waiting time of 1. P0 then executed with the largest burst time in the set ($BT = 5$) pushing the completion time up to 12. Because P0 occupied the CPU for so long, the two processes that arrived shortly after it, P2 ($AT = 4$) and P4 ($AT = 5$), had to wait 8 and 10 time units respectively before getting the CPU, even though their own burst times were relatively short (3 units each).

A single long process (P0) that happens to be scheduled early forces every subsequent short process to queue behind it increasing their individual and overall waiting times. The computed Average Turnaround Time was 8.00 and the Average Waiting Time was 4.60, both of which are noticeably higher than they would be if the shorter processes (P2, P3, P4) had been scheduled ahead of the longer one (P0). This highlights a key limitation of FCFS: because it only considers arrival order and ignores burst time, it cannot prevent short jobs from being delayed behind long ones.

5. Conclusion:

The FCFS scheduling algorithm was successfully implemented and tested on a new set of five processes. The execution order strictly followed the arrival times and all Completion Time, Turnaround Time, and Waiting Time, were computed correctly, yielding an Average Turnaround Time of 8.00 and an Average Waiting Time of 4.60. The results reinforce that while FCFS is simple to implement it is highly sensitive to the burst time of whichever process happens to arrive early. In this run, the long burst time of P0 substantially increasing the waiting times of the shorter processes P2 and P4 that arrived soon after it. This confirms that

FCFS does not guarantee an optimal average waiting time and can perform poorly whenever a long process precedes several shorter ones in the arrival sequence.

6. Github:

[Click here to view source code](#)